FinTwin

AI-Powered Financial Health Digital Twin

A beginner's guide to how the website works, how it was built, where its data comes from, and how to explain it in a viva.

What it is	A web app that builds a live model (a "digital twin") of a person's finances and uses it to analyze, predict, simulate and explain.
Built with	React + Vite (frontend) · Flask + Python (backend) · PostgreSQL (database) · pandas, scikit-learn, XGBoost (ML) · optional OpenAI/Gemini (LLM)
Based on	The project's source code, configuration, tests, git history and documentation (repository state of 10 October 2026, commit c140001).
Who it's for	Anyone with basic programming knowledge who needs to understand, demonstrate and explain FinTwin.

How to read this guide. Each chapter explains first *what the user sees*, then *what happens behind the scenes*. Boxes marked **VERIFIED** state facts checked directly in the code or by running it; boxes marked **NOT VERIFIED** state what could not be confirmed. No passwords, keys or tokens appear anywhere in this guide.

FinTwin is a decision-support and educational tool, not a licensed financial advisor. Forecasts and simulations are estimates.

Table of Contents

1. Project Overview . . . 4

1.1 Purpose . . . 4

1.2 Problem statement . . . 4

1.3 Objectives . . . 4

1.4 Real-world applications . . . 5

2. Features and Functionality . . . 6

2.1 Accounts and the demo account . . . 6

2.2 Profile, investments and loans . . . 6

2.3 Transactions and CSV import . . . 6

2.4 Dashboard . . . 7

2.5 Digital Twin page . . . 7

2.6 Spending analytics . . . 7

2.7 Financial Health Score . . . 7

2.8 Forecasts . . . 8

2.9 Risks and alerts . . . 8

2.10 Goals . . . 8

2.11 What-If Simulator . . . 8

2.12 AI Financial Coach . . . 9

3. Technology Stack . . . 10

3.1 Languages . . . 10

3.2 Frontend . . . 10

3.3 Backend . . . 10

3.4 Data, machine learning and AI . . . 11

3.5 Database and infrastructure . . . 11

4. How the Website Is Built . . . 12

4.1 The big picture . . . 12

4.2 Development process . . . 12

4.3 Frontend . . . 13

4.4 Backend . . . 13

4.5 Database . . . 13

4.6 Authentication . . . 13

4.7 API integrations . . . 14

4.8 How the components communicate . . . 14

5. Codebase Structure . . . 15

5.1 The services folder (the "brain") . . . 15
5.2 Important single files . . . 16
6. Data Sources and the Demo Account . . . 17
6.1 The verdict at a glance . . . 17
6.2 How it is created . . . 17
6.3 How it is stored, retrieved and displayed . . . 18
6.4 How demo data differs from a real user's data . . . 18
6.5 Other data sources in the project . . . 19
7. End-to-End Workflow . . . 20
7.1 Logging in . . . 20
7.2 Submitting information (example: saving the profile) . . . 20
7.3 Building the twin for a page . . . 21
7.4 Running a What-If simulation . . . 21
7.5 Asking the AI coach . . . 22
7.6 What happens when something fails . . . 22
8. Database and APIs . . . 24
8.1 Database schema . . . 24
8.2 API endpoints . . . 25
8.3 Request and response format . . . 25
8.4 Data flow summary . . . 26
8.5 External integrations . . . 26
9. Deployment and Hosting . . . 27
9.1 Hosted architecture . . . 27
9.2 How deployment works . . . 27
9.3 Environment variables . . . 28
9.4 How the frontend and backend are connected . . . 28
10. Limitations and Future Improvements . . . 29
10.1 Incomplete or missing features . . . 29
10.2 Technical limitations . . . 29
10.3 Security: what is implemented, and the concerns . . . 29
10.4 Possible improvements . . . 30
11. Technical Glossary . . . 31
12. Project Viva Questions and Answers . . . 33
12.1 A five-minute demonstration script . . . 36

1. Project Overview

1.1 Purpose

FinTwin is a web application that helps a person understand and plan their money. It builds a **Financial Digital Twin**: a live, calculated model of the user's finances (income, spending, savings, investments, loans and goals), rebuilt from their real data whenever that data changes. Every feature of the website works on this twin. Its slogan sums up the four jobs it does: **Analyze. Predict. Simulate. Decide Better.**

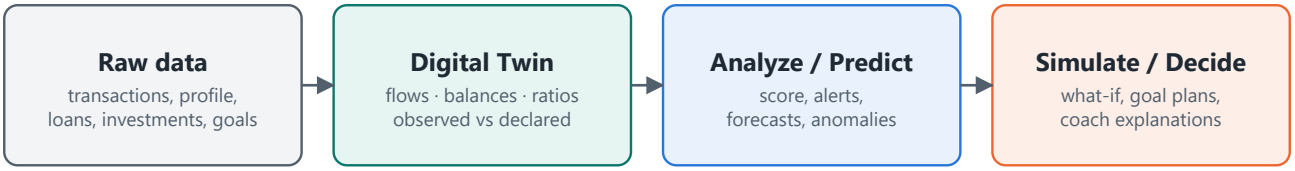


Figure 1. The twin sits between raw data and every feature.

1.2 Problem statement

Most money tools look backwards. A bank app or an expense tracker answers "*what did I spend?*", but not the questions people actually face before a decision:

- Am I financially healthy, and **why** did that change since last month?
- Where is my spending heading over the next few months?
- **What happens if** I buy a ₹1.3 lakh laptop, take a loan, or my income drops by 20%?
- Is my goal (a car, a trip, higher studies) realistic, and what would make it realistic?

General chatbots are not a safe answer either: they do not know the user's data, and they can state invented numbers confidently. A financial tool cannot afford wrong numbers.

1.3 Objectives

1. Turn raw transactions into a clear, always-current picture of the user's finances (the twin).
2. Score financial health from 0 to 100 using transparent, published personal-finance guidelines, and explain every point.
3. Predict future spending with machine learning, and show how uncertain the prediction is.
4. Detect risks and unusual transactions automatically.
5. Let users test decisions safely with a deterministic What-If simulator before spending money.
6. Plan goals: required monthly saving, feasibility, and concrete options.
7. Offer an AI coach that explains results in plain English **without ever inventing numbers**.
8. Degrade gracefully: the app must keep working when data is short, a model is missing, or an AI service is unavailable.

1.4 Real-world applications

Who	How FinTwin-style software helps
Individuals / young earners	Budget health check, "can I afford it?" decisions, goal planning, spotting subscription or food-delivery creep.
Banks and fintech apps	Personal-finance-management (PFM) features: insights, nudges and affordability checks built on a customer's transaction history.
Financial-literacy education	A safe sandbox to show how EMIs, SIPs, emergency funds and compounding affect a household budget.
Advisers and counsellors	A shared, explainable picture of a client's finances before giving advice (FinTwin itself does not give regulated advice).

NOTE FinTwin is an educational and decision-support tool. It is not connected to any bank, and it is not a licensed financial adviser; the app shows this disclaimer on every page.

2. Features and Functionality

The website has a public landing page, login and registration pages, and ten pages inside the app, reached from the left sidebar. This chapter explains each major feature in the same three-part way.

2.1 Accounts and the demo account

What the user sees	How to use it	What happens behind the scenes
Landing page, Log in, Register, and an **Explore the demo account** button.	Register with name, email and a password (8+ characters with a letter and a digit), or click the demo button.	The API stores a scrypt **hash** of the password (never the password), creates an empty financial profile and returns a **JWT** token. The demo button calls `POST /api/auth/demo`, which returns a token for the seeded demo user, so the demo password never reaches the browser.

2.2 Profile, investments and loans

What the user sees	How to use it	What happens behind the scenes
Profile & Settings page: declared income and expenses, emergency fund, bank balance (with a date), risk tolerance, employment, dependents; lists of investments and loans; ML model status.	Fill in the form and click **Save profile**; add or edit holdings with the Add buttons.	Validated by pydantic schemas, saved to `financial_profiles`, `investments` and `loans`. After every save an *after-request hook* rebuilds the twin, the health-score history and the risk alerts.

2.3 Transactions and CSV import

What the user sees	How to use it	What happens behind the scenes
A filterable, paginated list; add / edit / delete; an **Import CSV** wizard with a per-row status (ok, warning, duplicate, error) and a **Download a sample statement** link.	Upload a bank statement (.csv, up to 2 MB), review the preview, untick or re-categorize rows, then confirm. Changing a category on any transaction teaches FinTwin for next time.	The preview endpoint parses dates and ₹ amounts, validates each row, auto-categorizes it (your past corrections → keyword rules → ML model) and flags duplicates, **without saving**. The confirm endpoint re-validates and saves all rows in one transaction (all-or-nothing).

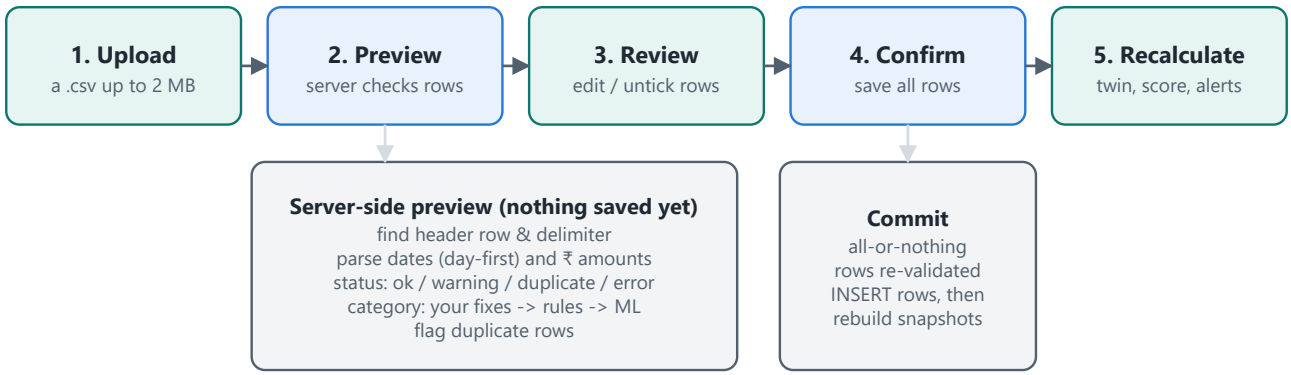


Figure 2. The two-step CSV import: nothing is written until the user confirms.

2.4 Dashboard

What the user sees	How to use it	What happens behind the scenes
Greeting, Financial Health Score gauge with seven component bars, six KPI tiles (cash, income, spending, surplus, savings ratio, emergency fund), a 12-month income vs spending chart, alerts, goals, a spending forecast and a **Quick what-if** box.	Open it after logging in; type an amount in Quick what-if and click Simulate.	One request, `GET /api/dashboard`, builds the twin once and returns the twin, health score, trends, active alerts, goal plans and the forecast together.

2.5 Digital Twin page

What the user sees	How to use it	What happens behind the scenes
A top-to-bottom story: data sources (observed vs declared) → indicator tiles → health score with a "how it's calculated" table and ranked improvements → predicted state → "try a scenario" cards.	Read from top to bottom; click a scenario card to open the simulator pre-filled.	`GET /api/digital-twin` builds a `TwinState` object from the database rows (see chapter 4). Each figure is labelled *observed* (from transactions) or *declared* (from the profile, used until enough history exists).

2.6 Spending analytics

What the user sees	How to use it	What happens behind the scenes
Range buttons (7 days to 12 months, or custom dates), totals compared with the previous period, essential vs discretionary split, category bars, income vs spending, running cash balance, category changes and top merchants.	Pick a range; hover charts for exact values.	pandas aggregations over the user's transactions for the chosen range (`/api/analytics/overview, /categories, /trends, /merchants`).

2.7 Financial Health Score

What the user sees	How to use it	What happens behind the scenes
A 0–100 score with a grade (A Excellent to E Critical), seven component scores, strengths, weaknesses, recommendations, and "why did my score change?".	Shown on the dashboard and twin page; the coach can also explain it.	Each component converts one measurable ratio to 0–100 using a curve anchored on guidelines (e.g. the 50/30/20 rule, 3–6 months of emergency fund, 36% debt-to-income). The total is a weighted mean; missing components are excluded rather than punished.

Component	Weight	Measures	Example anchor
Savings	20	share of income left after spending and EMIs	20% saved → 80 points
Emergency fund	20	months of essentials + EMIs covered	3 months → 70, 6+ → 100
Debt burden	20	EMIs ÷ income	36% → 50 points
Cash-flow stability	15	positive months + how much spending varies	—
Investments	10	investment rate + number of asset types	10% of income → 70

Component	Weight	Measures	Example anchor
Goal progress	10	share of goals' required saving the surplus can fund	priority-weighted
Spending behaviour	5	discretionary share + recent serious anomalies	≤ 30% → 100

Table 1. Health-score components (from services/financial/health_score.py).

2.8 Forecasts

What the user sees	How to use it	What happens behind the scenes
A forecast chart with an 80% "likely range" band, a per-category table, the model that was chosen and its backtest error, and a cash-flow projection.	Choose a horizon (1–6 months for spending, up to 12 for cash flow).	Five models compete: four simple baselines and a **global XGBoost** model trained offline. XGBoost is used unless a baseline was at least 20% more accurate on this user's own last 6 months ("champion–challenger").

2.9 Risks and alerts

What the user sees	How to use it	What happens behind the scenes
Alerts with a severity badge (LOW, MEDIUM, HIGH, CRITICAL), an explanation and a recommended action; dismiss/restore; Re-scan.	Open Risks & Alerts; dismiss what you have dealt with.	Ratio rules (e.g. savings < 10%, EMIs > 36% of income, emergency fund < 3 months), unusual transactions (robust statistics + Isolation Forest), abnormal months, category drift and duplicate charges. Alerts carry a fingerprint so re-scans never duplicate them.

2.10 Goals

What the user sees	How to use it	What happens behind the scenes
Each goal with progress, required saving per month, what you plan, what your surplus can fund, the projected completion date, status (on track, at risk, off track, …) and concrete options.	Click **New goal** and enter a target, amount saved, date, monthly contribution and priority.	Annuity formulas compute the monthly saving needed; the monthly surplus is allocated to goals in priority order. Options are recalculated (save more, free up spending, move the date, lower the target).

2.11 What-If Simulator

What the user sees	How to use it	What happens behind the scenes
Scenario editor with 8 change types, horizon and inflation; results with a risk level, impact tiles, a month-by-month chart, a current-vs-simulated table, goal delays and the full list of assumptions; saved scenarios.	Add one or more changes (e.g. Big purchase ₹90,000), click **Run simulation**.	A deterministic month-by-month cash-flow projection is run twice, without and with the changes, and the two futures are compared. Same inputs always give the same output (chapter 7.4).

2.12 AI Financial Coach

What the user sees	How to use it	What happens behind the scenes
A chat page with suggested questions; answers are split into *your data*, *calculations*, *estimates*, *what you could do* and *background*, with sources and the name of the engine that answered.	Ask in plain English, e.g. "Can I afford a laptop for 1.3 lakh?" or "Why did my score drop?"	Intent rules + amount parsing + a context builder that calls the same services as the app; affordability questions run the simulator; a retrieval step (RAG) adds guidance from 13 written guides; an optional LLM may phrase the answer, but a grounding check rejects any number not present in the data.

3. Technology Stack

Every technology below was confirmed in the repository's dependency files (`backend/requirements.txt`, `frontend/package.json`, Dockerfiles and `docker-compose.yml`).

3.1 Languages

Language	Where	Why
Python 3.11	Backend API, financial calculations, ML training	The standard language for data work and ML; pandas, scikit-learn and XGBoost are Python libraries.
JavaScript (JSX)	Frontend (React components)	Runs in every browser; JSX lets HTML-like markup live next to the code that controls it.
SQL (via SQLAlchemy)	Database queries and migrations	PostgreSQL speaks SQL; the ORM writes it for us, safely parameterised.
HTML / CSS (Tailwind)	Page structure and styling	Tailwind utility classes keep styling next to the component.

3.2 Frontend

Library / tool	Version	Used for	Why chosen
React	19	Building the user interface from components	Most widely used UI library; component model fits a dashboard.
Vite	8	Dev server and production build	Very fast; proxies `/api` to Flask in development.
React Router	8	Page URLs such as /dashboard, /goals	Standard routing for React single-page apps.
Recharts	3	All charts	Declarative React charts with tooltips; no canvas code needed.
Tailwind CSS	4	Styling	Fast, consistent design without separate CSS files.
lucide-react	1.x	Icons	Lightweight, consistent icon set.
oxlint	1.x	Linting (code quality checks)	Ships with the current Vite template; very fast.
Node test runner	built into Node	Unit tests for formatting helpers	No extra dependency.

3.3 Backend

Library	Version	Used for	Why chosen
Flask	3.1	Web framework: URLs → Python functions	Small and easy to understand; the project synopsis specified it.
Flask-SQLAlchemy / SQLAlchemy	3.1 / 2.x	Talking to the database with Python classes (ORM)	Safe parameterised queries; same models work on PostgreSQL and SQLite.
Flask-Migrate (Alembic)	4.1	Versioned database schema changes	Reproducible schema on any machine with one command.

Library	Version	Used for	Why chosen
Flask-JWT-Extended	4.7	Login tokens (JWT)	Stateless authentication suited to a separate frontend.
flask-cors	6.0	Allowing the frontend's domain to call the API	Needed when frontend and backend are on different domains (Vercel and Render).
pydantic	2.13	Validating request bodies	Clear error messages; rejects unknown fields.
psycopg	3.3	PostgreSQL driver	Modern driver for Python.
gunicorn	26	Production web server	Runs several Flask workers reliably.
pytest, pytest-cov, ruff	9 / 7 / 0.16	Tests, coverage, lint/format	Industry-standard Python tooling.

3.4 Data, machine learning and AI

Library / service	Used for	Why chosen
pandas, NumPy	Turning transactions into monthly tables; all aggregations; simulations	The standard tools for tabular data in Python.
scikit-learn	Logistic Regression + TF-IDF (categorizer), Isolation Forest (anomalies), TruncatedSVD (retrieval embeddings)	Reliable, well-documented classic ML.
XGBoost	Spending forecaster	Strong on tabular data with lag features; small and fast; evaluated against baselines (chapter 12).
joblib	Saving and loading trained models	Standard for scikit-learn/XGBoost objects.
OpenAI / Google Gemini APIs (optional)	Phrasing coach answers	Configurable by environment variable; called with Python's built-in urllib (no vendor SDK). Off by default.

3.5 Database and infrastructure

Technology	Used for	Why chosen
PostgreSQL 16	Main database	Reliable relational database; strong constraints and transactions.
SQLite	Automated tests and quick local try-outs	No server needed; tests run in memory.
Docker + docker compose	One-command local stack (database + API + web server)	Same environment on every machine.
nginx	Serves the built frontend and forwards /api in the Docker stack	Fast static file server and reverse proxy.
Vercel · Render · Neon	Prepared hosting: frontend · API · database (chapter 9)	Free tiers; deploy straight from GitHub.
Git + GitHub	Version control (14 commits, one per phase)	History of how the project was built.

4. How the Website Is Built

4.1 The big picture

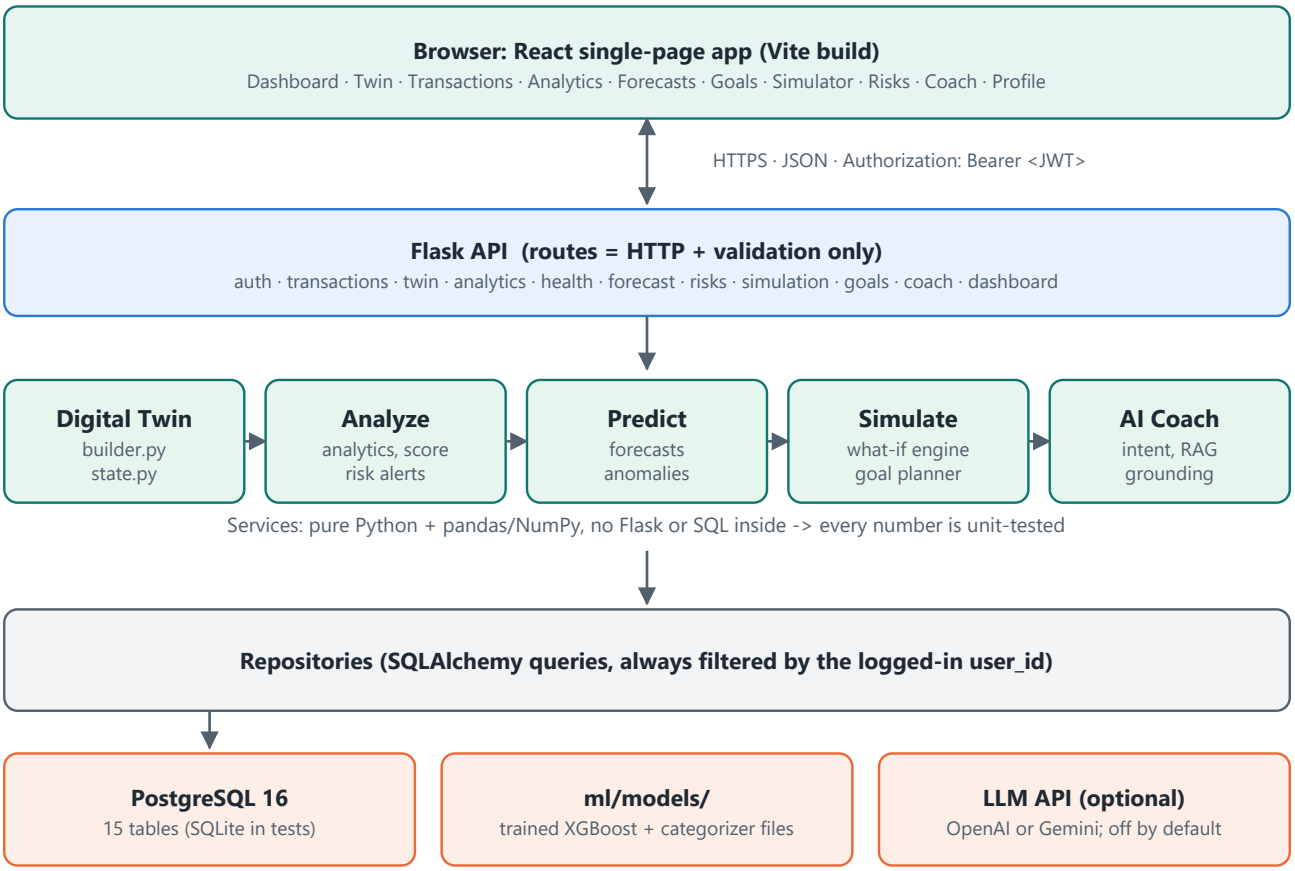


Figure 3. Layered architecture. Requests flow down; JSON answers flow back up.

One rule makes the system reliable: routes only handle HTTP and validation; *services* do all the calculations and never touch Flask or the database; *repositories* do all database access. Because the services are plain Python functions, every financial number can be unit-tested with hand-made data, without a server or a database.

4.2 Development process

The project was built in **14 phases**, each ending with tests, fixes, documentation and a git commit. The git history (verified) shows one commit per phase between 8 and 9 October 2026.

Phase	What was built
1–2	Requirements, architecture document, repository and development environment
3	Database schema, migration, synthetic demo data
4	Core APIs: authentication, profile, transactions, CSV import, investments, loans
5	Digital Twin builder, monthly snapshots, analytics
6	Financial Health Score
7	Forecasting, anomaly/risk detection, ML categorizer, offline training pipeline
8–9	What-If simulation engine; goal planner

Phase	What was built
10	AI Financial Coach with retrieval (RAG) and grounding check
11	React frontend (all pages and charts)
12	End-to-end test of all 18 acceptance criteria as a new user, with fixes
13	Testing hardening: every API route with junk input, calendar edge dates, CSV edge cases
14	Docker deployment, README, viva guide; then hosting configuration for Vercel + Render + Neon

4.3 Frontend

- **Single-page app:** the browser downloads the app once; moving between pages does not reload the page. App pages are *lazy-loaded*, so the chart library downloads only when it is needed.
- **services/api.js** is the only place that talks to the backend. It adds the login token, unwraps the `{data, meta}` response, turns errors into readable messages, and logs the user out on a 401 (invalid or expired token).
- **hooks/useApi.js** loads data for a page and exposes `loading`, `error` and `reload`, so every page can show a spinner, an error with "Try again", or the data.
- **context/AuthContext.jsx** remembers who is logged in. On reload it calls `/api/auth/me`; only a 401 logs the user out, so a temporary server outage does not.
- **ErrorBoundary** around each page keeps the sidebar alive if one page crashes.

4.4 Backend

- **app/__init__.py** creates the Flask app (the "app factory"), registers 14 blueprints under `/api`, sets up CORS, JWT and error handlers.
- **Blueprints** group URLs by area (auth, transactions, twin, analytics, health, forecast, risks, simulation, goals, coach, dashboard, profile, holdings, system). 56 routes in total (verified by listing the URL map).
- **Validation:** request bodies are parsed into pydantic models that reject unknown fields and give field-level errors.
- **Recalculation:** every successful write in a data blueprint triggers `recalculate()`, which rebuilds monthly snapshots, health scores and risk alerts. A failure here is logged and never undoes the user's write.
- **Error envelope:** every error, including crashes, is returned as `{"error": {"code", "message", "details"}}` with the right HTTP status.

4.5 Database

PostgreSQL stores 15 tables (chapter 8). The schema is created by an Alembic migration (`database/migrations`) with `flask db upgrade`. Money is stored as exact decimals (`NUMERIC(14,2)`), and constraints protect data quality (e.g. a transaction amount must be greater than zero).

4.6 Authentication

- Passwords are hashed with **scrypt** (Werkzeug's default; verified by generating a hash).
- Login returns a **JWT access token** valid for 12 hours, signed with `JWT_SECRET_KEY`. The app refuses to start in production with the default secret.
- Every protected request sends `Authorization: Bearer <token>`; the API reads the user id from the verified token, never from the request body.
- Requests for another user's data return 404 "not found", so the API never reveals that the record exists.

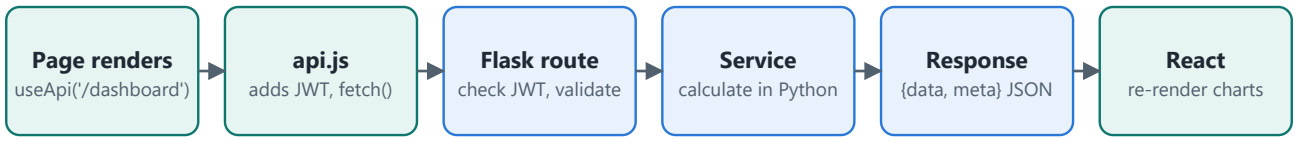
- An unknown email still runs a password check against a dummy hash, so response time does not reveal which emails are registered.

4.7 API integrations

The only external APIs are the **optional** LLM providers: OpenAI (`api.openai.com/v1/chat/completions`) and Google Gemini (`generativelanguage.googleapis.com`), selected by `LLM_PROVIDER`. The default is `local`, which calls no external service. There is **no bank API**: financial data enters only by manual entry, CSV upload or the seed command.

> **NOT VERIFIED** Live calls to OpenAI or Gemini were never made during development (no API key was available). They are covered only by tests that simulate the API responses.

4.8 How the components communicate



Errors come back as {error: {code, message, details}} and are shown as a readable message with 'Try again'.

Figure 4. The life of one request, from page to chart.

In development the Vite dev server forwards `/api` to Flask on port 5000. In the Docker stack nginx does the same. When the frontend and backend are hosted on different domains (Vercel and Render), the frontend is built with `VITE_API_URL` pointing at the API, and the API's CORS setting allows the frontend's domain.

5. Codebase Structure

The repository has about 8,900 lines of backend Python, 4,200 lines of backend tests and 3,000 lines of frontend JavaScript (counted with `wc -l`).

```
Fintwin/
├── backend/                Flask API
│   ├── app/
│   │   ├── routes/         URLs (one file per area)
│   │   ├── schemas/        request validation (pydantic)
│   │   ├── models/         database tables (SQLAlchemy)
│   │   ├── repositories/   database queries, always per user
│   │   ├── services/       all calculations (see table below)
│   │   ├── seed/           demo user, synthetic data, sample CSV
│   │   ├── utils/          auth, dates, money formatting, HTTP envelope
│   │   ├── cli.py          flask seed · flask sample-csv · flask ml train
│   │   └── config.py       settings read from environment variables
│   ├── knowledge_base/     13 guides used by the coach's retrieval
│   ├── tests/              pytest suite (503 tests)
│   └── Dockerfile, docker-entrypoint.sh, requirements.txt, run.py
├── frontend/               React app
│   ├── src/pages/          one file per page
│   ├── src/components/     shared UI pieces (cards, badges, gauge, modal…)
│   ├── src/charts/         all Recharts charts
│   ├── src/services/api.js  the only code that calls the API
│   └── Dockerfile, nginx.conf, vercel.json, package.json
├── ml/                     offline training pipeline + metrics
├── database/               migrations/ (Alembic) · seed/ (sample CSV)
├── docs/                   architecture, api, ml, simulation, setup, viva
├── docker-compose.yml      database + API + web server in one command
└── render.yaml             Render hosting blueprint for the API
```

5.1 The services folder (the "brain")

Folder / file	Responsibility
digital_twin/builder.py	Builds the TwinState from raw data; `derive()` holds every ratio formula in one place
digital_twin/service.py	Snapshots per month, recalculation after writes, health report
financial/health_score.py	The 0–100 score, grades, explanations and "why did it change"
financial/goals.py, goal_planner.py	Surplus allocation by priority; goal plans and options
financial/formulas.py	EMI, outstanding loan balance, future value of SIPs
analytics/aggregations.py	Date ranges, totals, categories, trends, merchants
analytics/anomalies.py, risk_service.py	Unusual transactions, abnormal months, drift; alerts
forecasting/models.py, service.py	Baselines + XGBoost; model selection; likely ranges
simulation/engine.py, scenarios.py, service.py	Month-by-month projection; 8 change types; impact and risk
coach/*	intent.py, entities.py, scenario_parser.py, context.py, rag.py, grounding.py, local.py, llm/
ml/features.py, categorizer.py, artifacts.py	Shared ML features; categorization cascade; model loading
importer/csv_import.py	Parsing and validating bank-statement CSV files

5.2 Important single files

File	Why it matters
backend/app/config.py	All settings come from environment variables; production safety checks
backend/app/utils/http.py	The JSON envelope and every error handler
backend/app/seed/demo.py	Creates the demo account (chapter 6)
backend/app/seed/synthetic.py	The synthetic transaction generator
ml/pipeline.py	Builds the synthetic population, trains, evaluates and saves models + metrics
frontend/src/App.jsx	All page routes, lazy loading, login guards
frontend/src/context/AuthContext.jsx	Login state, session restore, logout
docker-compose.yml, render.yaml, vercel.json	How the app runs locally and where it is hosted

6. Data Sources and the Demo Account

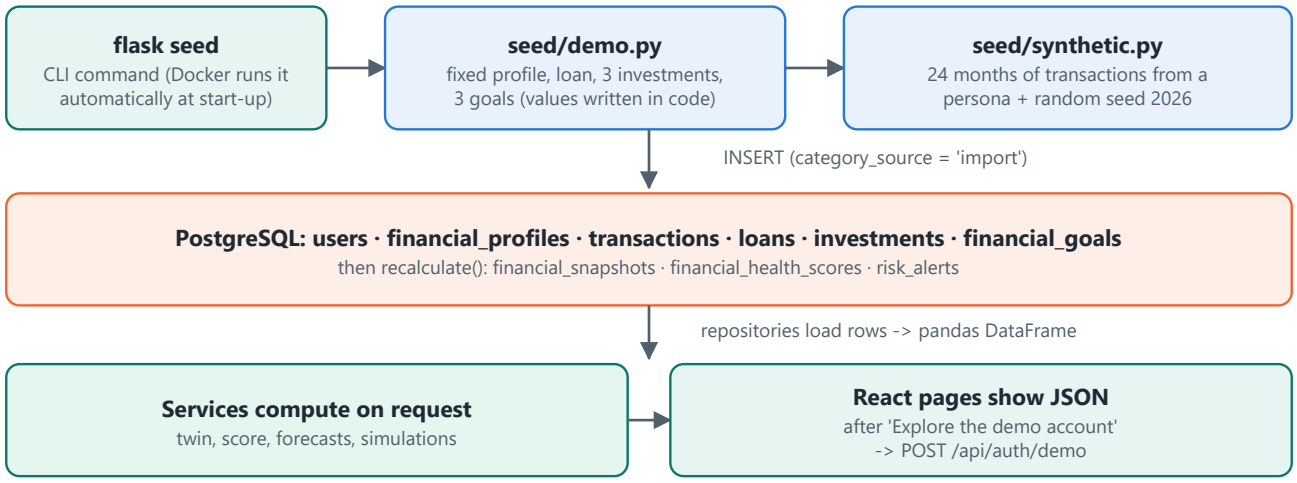
This chapter answers a common viva question precisely: **where does the demo account's data come from?** Every statement here was checked in `backend/app/seed/demo.py`, `backend/app/seed/synthetic.py`, `backend/app/cli.py` and `backend/docker-entrypoint.sh`.

> **VERIFIED** The demo data is **synthetic** (computer-generated), **stored in the database**, and served by the same API as real users' data. It is **not** fetched from any external API, and no demo figures are hard-coded in the frontend. Some starting values (the loan, investments, goals and profile) are fixed constants in the seed code; the transactions are generated.

6.1 The verdict at a glance

Possible source	Used for the demo?	Evidence
Hard-coded in the frontend	No	All pages render API responses; the demo button only calls `POST /api/auth/demo`.
Hard-coded constants in backend seed code	Partly	Profile, 1 loan, 3 investments, 3 goals are written as fixed values in `seed/demo.py`.
Generated synthetically	Yes (all transactions)	`generate_transactions(demo_persona(), today, 24, seed=2026)` in `seed/demo.py`.
Stored in a database	Yes	Rows are inserted into PostgreSQL tables and read back by repositories.
Retrieved from an external API	No	The seed code makes no network calls; there is no bank API.
Real people's data	No	The persona ("Aarav Sharma") is fictional; merchants are typical Indian names.

6.2 How it is created



Nothing about the demo is fetched from an external API, and no figures are hard-coded in the frontend.

Figure 5. From the seed command to the screen.

1. **Trigger.** The command `flask seed` runs it. The Docker image runs this command automatically every time the API container starts (`docker-entrypoint.sh`), unless `SEED_DEMO=false`.

2. **Idempotent.** If a user with the demo email already exists, nothing is changed. `flask seed --reset` deletes the demo user (and, by cascade, all their data) and creates it again.
3. **Categories.** 22 shared categories (Salary, Groceries, Food & Dining, EMI/Debt…) are created if missing.
4. **User and profile.** A user named Aarav Sharma (email from `DEMO_EMAIL`, password from `DEMO_PASSWORD`, stored only as a hash) with a fixed profile: declared income ₹95,000, emergency fund ₹95,000, opening balance ₹25,000 dated at the start of the history, one dependent, salaried, medium risk tolerance.
5. **Transactions (synthetic).** 24 calendar months ending on the day the seed runs, from a *persona* (salary ₹95,000 with an 8% raise after a year, annual bonus, rent ₹26,000, ₹8,000 monthly family support, groceries, food delivery, rides, shopping, utilities, a June health-insurance premium, December travel, festive Oct/Nov shopping, a ₹10,000 SIP and the loan EMI). Amounts are drawn from random distributions with a **fixed random seed (2026)**, so the same date gives the same data.
6. **Planted events.** Four realistic irregularities are injected so that anomaly detection has something to find: a travel-heavy month (9 months before the end), a hospital bill (7), a large electronics purchase (4) and a duplicate charge (2). Food-delivery spending also creeps upward in the final months. The generator's ground-truth labels for these are **not** stored in the database.
7. **Holdings and goals (fixed values).** HDFC Education Loan ₹6,00,000 at 10% for 72 months, started 6 months before the history (its outstanding balance is computed with the amortisation formula); Axis Bluechip Fund SIP ₹10,000/month (value computed with the future-value formula); Public Provident Fund (₹1,50,000 invested, ₹1,68,400 now); Sovereign Gold Bond (₹40,000 invested, ₹48,200 now); goals "Buy a car" ₹8,00,000, "Europe trip" ₹3,00,000 and "Higher studies fund" ₹5,00,000, dated 24, 18 and 36 months ahead.
8. **Derived data.** Finally `recalculate()` stores monthly snapshots, health scores and risk alerts for the new user.

When the seed ran on 9 October 2026 it logged **1,161 transactions**. The exact count depends on the date, because the history always ends on the day of seeding.

6.3 How it is stored, retrieved and displayed

- **Stored** in the same tables as everyone else's data: users, financial_profiles, transactions (with `category_source = 'import'`), loans, investments, financial_goals, plus the derived snapshot, score and alert tables.
- **Retrieved** by repositories (e.g. `load_transactions(user_id)`), which run SQL filtered by the logged-in user's id and convert rows into a pandas DataFrame.
- **Calculated** on each request by the services (twin, score, forecasts, simulations).
- **Displayed** by React pages, which only render the JSON the API returns.

6.4 How demo data differs from a real user's data

Aspect	Demo account	A real (registered) user
How the account is created	By `flask seed`	By the Register page
How you log in	One-click button (`/api/auth/demo`), when enabled	Email + password
Where transactions come from	Synthetic generator	Manual entry or the user's CSV upload
Category source label	`import` for every seeded row	`manual`, `rule`, `ml`, `memory` or `import`
Dates	Last 24 months up to the seeding day; not refreshed automatically	Whatever the user enters

Aspect	Demo account	A real (registered) user
Shared?	Yes: every visitor who uses the demo button opens the **same** account	Private to the user
Tables, code, calculations	Identical	Identical

CAUTION Because the demo is one shared account, a visitor can add or delete its data and others will see the change. Run `flask seed --reset` to restore it. Also, as days pass after seeding, the demo history stops at the seeding day, so "last 30 days" views gradually become empty until it is reset.

6.5 Other data sources in the project

Source	What it is	Stored?
Sample statement download	`GET /api/transactions/import/sample` generates a 6-month CSV for a different fictional persona (Brightwave Software, salary ₹72,000), for trying the import	Only if the user imports it
database/seed/sample_bank_statement.csv	A small fixed CSV with one duplicate and two invalid rows on purpose, to demonstrate validation	No (file in the repository)
ML training population	300 fictional personas × 30 months (~404,000 transactions) generated in memory by `ml/pipeline.py` to train and evaluate models	No (only the trained model files and metrics are saved)
Categorizer corpus	`ml/datasets/categorizer_corpus.csv`: 349 hand-curated Indian merchant names with categories	File in the repository
Knowledge base	13 short guides written for the project (`backend/knowledge_base`)	Files; indexed in memory

7. End-to-End Workflow

This chapter follows real requests through the system, step by step.

7.1 Logging in

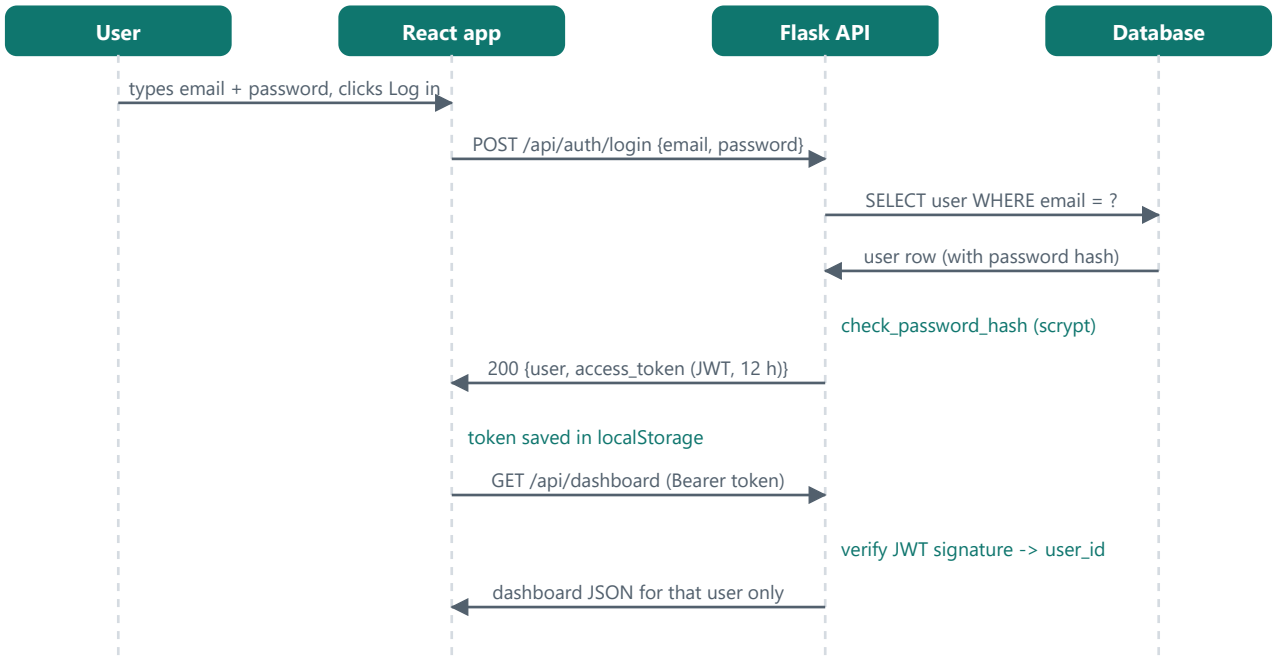


Figure 6. Login and the first authenticated request.

1. The user submits the login form. React calls `POST /api/auth/login`.
2. Flask validates the body (a valid email, a password), finds the user, and checks the password against the stored scrypt hash.
3. On success it returns the user and a signed JWT valid for 12 hours. On failure it returns 401 "Incorrect email or password" (the same message whether the email or the password was wrong).
4. React stores the token in the browser's localStorage and opens the dashboard.
5. Every later request carries the token. Flask verifies its signature and reads the user id from it; every database query is filtered by that id.
6. When the page is reloaded, React calls `GET /api/auth/me` to restore the session. If the token has expired the user is sent to the login page; if the server is merely unreachable, a "Cannot reach the FinTwin server" message with **Try again** is shown and the session is kept.

7.2 Submitting information (example: saving the profile)

1. The user edits the Profile form and clicks **Save profile**.
2. React sends `PUT /api/financial-profile` with the changed fields as JSON.
3. pydantic validates the body (non-negative amounts, known options, no unknown fields). Invalid input returns 400 with field-by-field messages that the form shows.
4. The profile row is updated and committed.
5. The **after-request hook** runs `recalculate(user_id)`: it reloads all the user's data, rebuilds the twin as of every past month-end and today, stores snapshots and health scores, and re-syncs risk alerts.
6. React shows "Saved — your twin was recalculated". Other pages show the new numbers the next time they load.

VERIFIED In the Phase 12 walkthrough, a new user saved a ₹60,000 emergency fund and an ₹85,000 bank balance. The dashboard then showed cash ₹85,000, 1.8 months of emergency cover and a health score that rose from 62 to 71, and the emergency-fund alert fell from HIGH to MEDIUM.

7.3 Building the twin for a page

When a page such as the dashboard loads, the backend does the following in well under a second:

1. **Load** the user's profile, transactions, loans, investments and goals (repositories → pandas).
2. **Average** income and spending over the last 3 *complete* calendar months (the current, partial month would understate spending). With no complete month yet, the declared profile values are used.
3. **Compute balances:** cash = opening balance + all income − all expenses since the opening date; investments and loan balances from their stored values.
4. **Derive ratios** in one function, `derive()`: savings ratio, debt-to-income, emergency months, investment rate, net worth.
5. **Score, detect, forecast** as each page needs, and return JSON.

```
Example (verified by unit test): income ₹1,00,000, spending ₹60,000, no EMIs
  savings ratio  = (100000 − 60000) / 100000 = 40%   → savings component 100
  emergency fund = ₹2,40,000 / ₹40,000 essentials   = 6 months → component 100
```

7.4 Running a What-If simulation

1. The user adds a change, e.g. a big purchase of ₹90,000, and clicks Run simulation (`POST /api/simulation/what-if`).
2. The engine copies the twin's monthly flows and balances as the **baseline**.
3. It projects month by month: cash += income − spending − EMIs − SIPs − goal contributions; investments grow at their recorded return; loans amortise. If cash goes negative the emergency fund covers it; beyond that it records a **shortfall**.
4. It runs the same projection again **with** the change, then compares the two at 12 months: cash, net worth, surplus, health score (using the same scoring code) and goal dates.
5. It sets a risk level by rule: any shortfall → CRITICAL; emergency cover < 1 month, EMIs > 50% of income or a score drop ≥ 15 → HIGH; a score drop ≥ 5, use of the emergency fund, a goal delayed ≥ 3 months or cash below one month of outgoings → MEDIUM; otherwise LOW.
6. The response lists every assumption, and React draws the comparison.

VERIFIED Phase 12 example (new user with ₹80,434 cash and a ₹8,949 monthly surplus): a ₹90,000 purchase was CRITICAL because cash ran short by ₹617 in the first month. The numbers reconcile: cash change −₹54,721 + goal-savings change −₹35,279 = −₹90,000.

7.5 Asking the AI coach

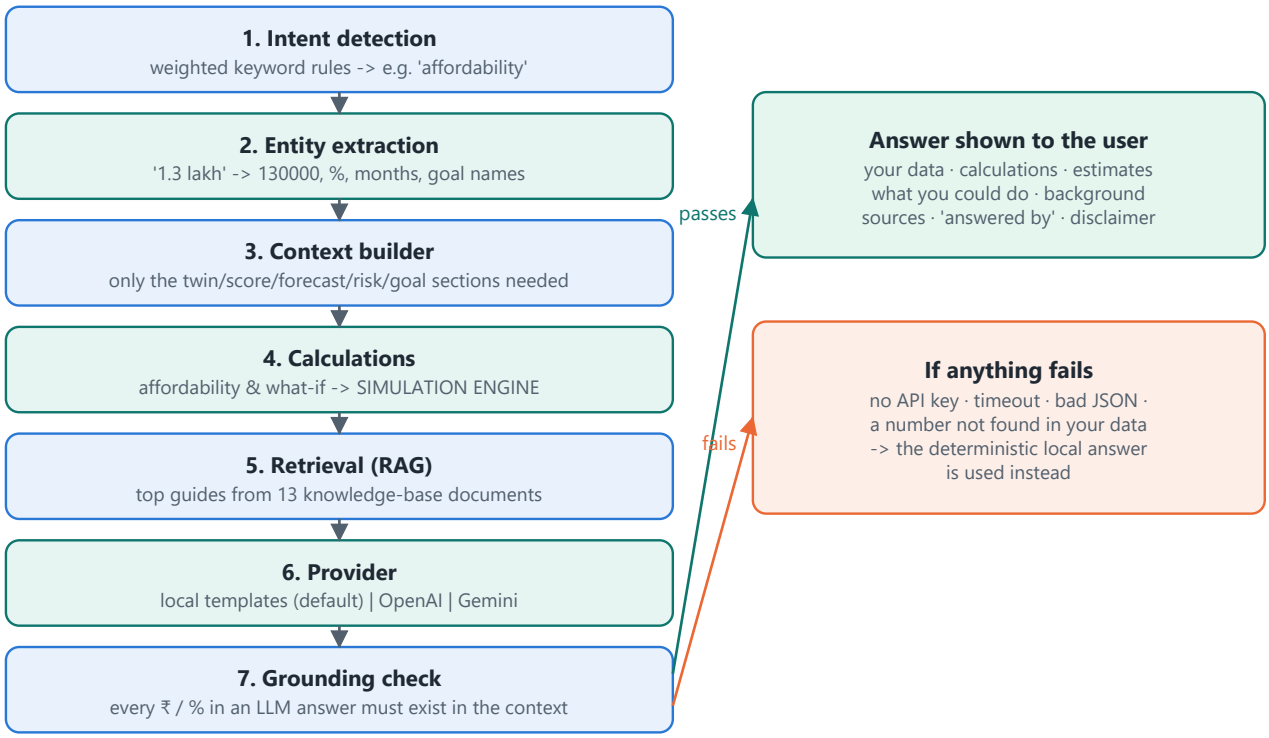


Figure 7. The coach pipeline. The language model, if used, only phrases verified numbers.

Example: "Can I afford a laptop for 1.3 lakh?" → intent *affordability*; amount ₹1,30,000; the simulator runs a ₹1,30,000 purchase; the answer reports the risk level (LOW for the demo account when checked), cash after 12 months with and without the purchase, goal delays, and a background note from the guide "Big purchases: affordability and timing".

7.6 What happens when something fails

Failure	What the system does	What the user sees
Backend down or restarting	Network error or proxy 5xx is detected by api.js	"Cannot reach the FinTwin server..." with Try again; still logged in
Database unavailable	Global handler returns 503 DB_UNAVAILABLE; /api/health reports "database: down"	An error message with Try again
Invalid input	400 VALIDATION_ERROR with field details	Messages next to the form
Token expired / invalid	401	Sent to the login page
Another user's record id	404 NOT_FOUND (existence not revealed)	Not found
ML model files missing	Forecasting uses statistical baselines; categorizer uses rules only; /api/ml/status says so	The app works; model labels change
LLM key missing, timeout, bad output, or an unverifiable number	Deterministic local answer is used	"Answered by FinTwin engine"
Too little history	Twin uses declared profile values; forecasts say "insufficient data" or "low confidence"	Notes explaining why

Failure	What the system does	What the user sees
Bad CSV rows	Row-level errors; good rows can still be imported	Per-row status in the wizard
Recalculation error after a save	Logged; the save itself is kept	Nothing breaks
A page crashes in the browser	ErrorBoundary catches it	That page shows an error; the menu still works

8. Database and APIs

8.1 Database schema

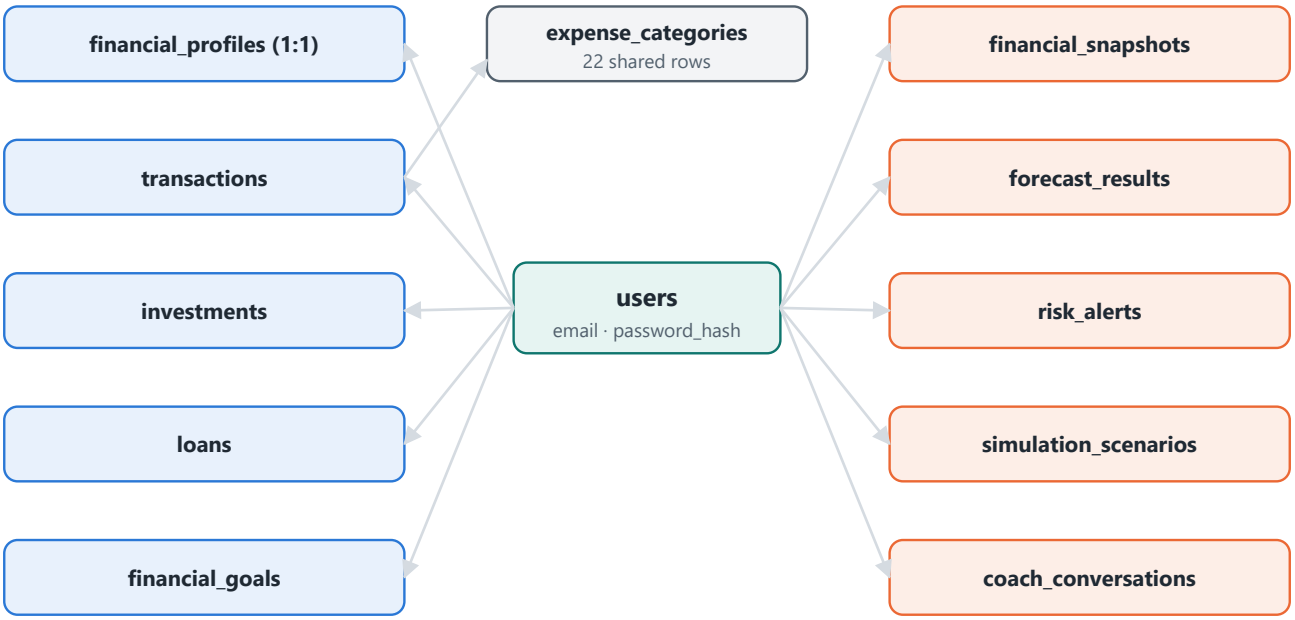


Figure 8. Twelve of the 15 tables; not drawn: financial_health_scores (one per snapshot), simulation_results (per scenario) and coach_messages (per conversation). Every user-owned table has user_id with ON DELETE CASCADE.

Table	Holds	Notes
users	email, password hash, full name	email is unique
financial_profiles	declared income/expenses, emergency fund, opening balance + date, risk tolerance, employment, dependents	one per user
expense_categories	22 shared categories with a kind: income, essential, discretionary, investment, debt	seeded
transactions	date, amount (> 0), type (income/expense), category, description, merchant, payment method, recurring flag, category source and confidence	income is a transaction too (one ledger)
investments	asset type (SIP, mutual fund, stocks, FD, PPF, gold, other), invested, current value, monthly contribution, expected return	
loans	principal, outstanding, interest rate, EMI, tenure, start date	outstanding ≤ principal
financial_goals	target, saved, target date, priority 1–5, monthly contribution, status	
financial_snapshots	the twin's key numbers at each month-end and on each change	unique per user, date and period
financial_health_scores	score, grade and the seven component scores	one per snapshot
forecast_results	model used, horizon, backtest error, forecast points (JSON)	
risk_alerts	type, severity, explanation, recommended action, status, fingerprint	fingerprint unique per user

Table	Holds	Notes
simulation_scenarios / simulation_results	saved scenarios (changes as JSON) and their results	
coach_conversations / coach_messages	chat history with the structured answer and provider	

Money columns use `NUMERIC(14,2)`. Option fields (e.g. severity) are text columns with CHECK constraints rather than PostgreSQL enum types, so adding a value never needs a type change. JSON columns are used only for genuinely variable data (forecast points, scenario changes, timelines, coach answers).

8.2 API endpoints

Base path `/api`. All routes except `/health`, `/auth/register`, `/auth/login` and `/auth/demo` require a token (verified by a test that calls every route without one).

Area	Endpoints
System	GET /health · GET /ml/status
Auth	POST /auth/register · POST /auth/login · POST /auth/demo · GET /auth/me
Profile and holdings	GET/PUT /financial-profile · GET/POST /investments, /loans, /goals · PUT/DELETE /investments/:id, /loans/:id, /goals/:id · GET /categories
Transactions	GET/POST /transactions · PUT/DELETE /transactions/:id · POST /transactions/import/preview · POST /transactions/import · GET /transactions/import/sample
Twin and score	GET /dashboard · GET /digital-twin · GET /digital-twin/history · GET /health-score · GET /health-score/history · GET /health-score/methodology
Analytics	GET /analytics/overview · /analytics/categories · /analytics/trends · /analytics/merchants
Forecasts and risks	GET /forecast/expenses · GET /forecast/cash-flow · GET /risks · POST /risks/scan · PATCH /risks/:id
Simulation	POST /simulation/what-if · GET/POST /simulation/scenarios · GET/DELETE /simulation/scenarios/:id · POST /simulation/scenarios/:id/run
Goals	GET /goals/plan · GET /goals/:id/plan · POST /goals/:id/simulate
Coach	POST /coach/chat · GET /coach/conversations · GET/DELETE /coach/conversations/:id

8.3 Request and response format

```
POST /api/simulation/what-if          Authorization: Bearer <token>
{ "type": "large_purchase", "amount": 130000 }

200 OK
{ "data": { "risk_level": "LOW", "explanation": "...", "impact": {...}, "timeline": [...],
            "assumptions": [...] },
  "meta": {} }

400 Bad Request (example)
{ "error": { "code": "VALIDATION_ERROR", "message": "Request validation failed.",
             "details": [ { "loc": ["amount"], "msg": "Input should be greater than 0" } ] } }
```

8.4 Data flow summary

- **In:** registration, profile form, holdings and goal forms, manual transactions, CSV import, seed command.
- **Stored:** raw data in its tables; derived history (snapshots, scores, alerts) refreshed after each write.
- **Computed on request:** twin, analytics, forecasts, simulations, goal plans, coach answers.
- **Out:** JSON to the React app; charts and tables render it.

8.5 External integrations

Integration	Status	Detail
OpenAI chat completions	Optional, off by default	Used only if LLM_PROVIDER=openai and a key is set
Google Gemini	Optional, off by default	Used only if LLM_PROVIDER=gemini and a key is set
Banks / Account Aggregator	Not implemented	Data enters by CSV or manual entry
Email, SMS, payments	Not implemented	No such features exist

9. Deployment and Hosting

FinTwin can run in three ways. They use the same code and differ only in configuration.

Mode	How	Status
Local development	PostgreSQL in Docker; `python run.py` (Flask on :5000); `npm run dev` (Vite on :5173, proxying /api)	Used throughout development
Docker, one command	`docker compose up --build` → PostgreSQL + API (gunicorn) + nginx serving the built frontend on http://localhost:8080	Verified on a clean copy of the repository, built with no cache
Hosted (free tiers)	Vercel (frontend) + Render (API in Docker) + Neon (PostgreSQL)	Configuration written and pushed; live deployment not verified (see box below)

NOT VERIFIED The hosting configuration (`render.yaml`, `frontend/vercel.json`) was committed and pushed to GitHub on 9 October 2026. Whether the site is currently live on Vercel and Render, and at which address, could not be checked from the repository: account sign-up and dashboard steps are performed by the project owner.

9.1 Hosted architecture

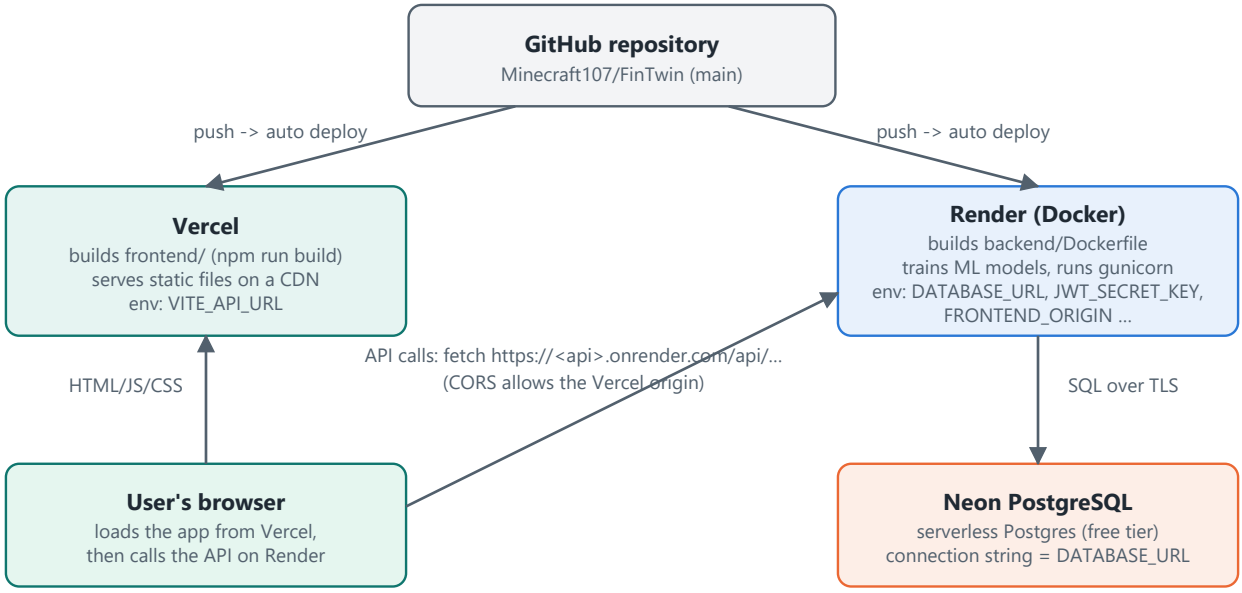


Figure 9. Vercel serves the frontend; the browser calls the API on Render; Render talks to Neon.

9.2 How deployment works

1. Code is pushed to the `main` branch on GitHub.
2. **Render** (blueprint `render.yaml`) builds `backend/Dockerfile`: installs Python packages, copies the code and trains the ML models into the image (about 100 seconds). On start, the container runs `flask db upgrade` (create/upgrade tables), `flask seed` (demo user, if missing) and gunicorn with 1 worker on the port Render provides. Render checks `/api/health`.
3. **Vercel** builds the `frontend` folder with `npm run build` and serves the static files from its CDN. `vercel.json` sends every page URL (e.g. /goals) to `index.html` so React Router can handle it.
4. **Neon** provides the PostgreSQL database; its connection string is given to Render as `DATABASE_URL`.

9.3 Environment variables

Settings and secrets are never written in the code; they are read from environment variables. Values are set in each host's dashboard (or a local `.env` file that git ignores). No secret values are shown here.

Variable	Where	Purpose
APP_ENV	Render	production enables production checks
JWT_SECRET_KEY	Render (generated)	Signs login tokens; the app refuses to start in production with the default
DATABASE_URL	Render	Database connection; postgres:// URLs are converted to the psycopg 3 driver automatically
FRONTEND_ORIGIN	Render	Which website(s) may call the API (CORS); comma-separated
DEMO_LOGIN_ENABLED, DEMO_EMAIL, DEMO_PASSWORD	Render	The one-click demo; the password is generated and never needed
GUNICORN_WORKERS, PORT	Render	1 worker fits the free 512 MB instance; PORT is set by Render
LLM_PROVIDER (+ OPENAI_/GEMINI_ keys)	Render	local by default; keys only if an LLM is wanted
VITE_API_URL	Vercel (build time)	The API address the frontend calls, e.g. https://<service>.onrender.com/api
POSTGRES_*, WEB_PORT, TRAIN_MODELS, SEED_DEMO	Docker compose	Local stack options

9.4 How the frontend and backend are connected

- The frontend is compiled with `VITE_API_URL`, so `api.js` sends every request to the Render address.
- The browser allows these cross-site calls only because the API answers with CORS headers naming the Vercel origin from `FRONTEND_ORIGIN`.
- Authentication works across domains because the token travels in the Authorization header, not in a cookie.
- In the Docker stack and in development there is only one origin (nginx or Vite forwards /api), so CORS is not involved.

> **NOTE** Render's free instance sleeps after 15 minutes without traffic; the next request takes about a minute while it wakes. Open the site a few minutes before a demonstration.

10. Limitations and Future Improvements

Everything below is based on the actual implementation (code, tests and documentation).

10.1 Incomplete or missing features

- No password reset, email verification or account deletion endpoints exist.
- No bank connection: data comes from CSV files or manual entry.
- No notifications (email/SMS/push) and no dark mode (deferred in the plan).
- Live OpenAI/Gemini calls are untested (mocked tests only); the local engine is the default.
- The AI coach's intent detection is rule-based, not a trained model (there was no real question data to train on).

10.2 Technical limitations

- **Synthetic training data.** Models are trained and evaluated on generated personas because no public labelled Indian household dataset exists. Results show the methods work as designed, not real-world accuracy.
- **Categorizer:** about 62% accurate on merchants it has never seen; pure brand names (e.g. "Zara") cannot be categorized from text. User corrections are remembered.
- **Forecasts:** income is held at its recent median; categories are forecast independently; ranges assume roughly normal errors. XGBoost results differ by about 1–2% between Windows and Linux.
- **Simulation:** no taxes, no market volatility, cash earns nothing; it shows consequences of stated assumptions, not a prediction.
- **Performance:** every write recalculates the whole history synchronously (about 0.1–0.4 s for 24 months) and refits the per-user anomaly model. Fine for one user at a time; a background job queue would be needed at scale.
- **Demo account:** shared by all visitors and not refreshed automatically (chapter 6.4).
- **Hosting:** free-tier cold starts (about a minute) and 512 MB memory.

10.3 Security: what is implemented, and the concerns

Implemented (verified in code)	Remaining concern
scrypt password hashing; same error and timing for unknown emails	No rate limiting on login (brute force)
JWT with 12-hour expiry; production refuses the default secret	Token stored in localStorage (exposed if an XSS bug ever existed); no refresh tokens or server-side logout
Every query scoped by the user id from the token; 404 for others' records	Demo login lets anyone open (and edit) the shared demo account when enabled
pydantic validation, unknown fields rejected; ORM queries (no SQL injection)	No audit log of changes
CSV size limit (2 MB in nginx and Flask), row limit 5,000, parsed as text, never executed	No virus scanning of uploads (only CSV text is accepted)
CORS limited to configured origins; secrets only in environment variables	No automated dependency vulnerability scanning
Docker container runs as a non-root user	HTTPS depends on the host (Vercel/Render provide it)

10.4 Possible improvements

- Retrain and re-evaluate on consented, anonymised real data; track live forecast error.
- Import through India's Account Aggregator framework instead of CSV files.
- Move recalculation to a background worker (e.g. a task queue) and cache results.
- httpOnly-cookie sessions with CSRF protection, refresh tokens, login rate limiting, password reset.
- Per-visitor demo accounts (a fresh copy per session) instead of one shared account.
- Monte-Carlo simulation for uncertain income and returns, alongside the deterministic engine.
- Sentence-transformer embeddings or pgvector for retrieval; a trained intent classifier.
- Tax-aware planning, insurance adequacy, retirement planning, household accounts, a mobile app.

11. Technical Glossary

Important terms in simple English, each with an example from FinTwin.

Term	Meaning	Example in FinTwin
API	Application Programming Interface: a set of URLs one program calls to talk to another.	The React app calls `GET /api/dashboard` to get dashboard data.
REST / endpoint	A style of API where each URL + method (GET, POST, PUT, DELETE) does one thing; each such URL is an endpoint.	`DELETE /api/goals/5` deletes goal 5.
JSON	A text format for data that both JavaScript and Python read easily.	`{"risk_level": "LOW"}`
Frontend / backend	Frontend = what runs in the browser (React). Backend = the server program (Flask) that holds the logic and data.	The gauge is frontend; the score calculation is backend.
Single-page app (SPA)	A website that loads once and swaps pages in the browser without full reloads.	Moving from Dashboard to Goals does not reload the page.
Component (React)	A reusable piece of UI written as a function.	`ScoreGauge` draws the score circle.
Framework / library	Ready-made code you build on. A framework calls your code; you call a library.	Flask (framework) calls your route functions; pandas (library) is called by you.
Blueprint (Flask)	A group of related URLs in Flask.	The goals blueprint holds all /api/goals routes.
ORM	Object-Relational Mapper: write Python classes instead of SQL; it writes safe SQL for you.	SQLAlchemy turns `Transaction(...)` into an INSERT.
Migration	A versioned script that creates or changes database tables.	`flask db upgrade` applies them.
Schema (database)	The design of tables, columns and rules.	transactions.amount must be > 0.
Foreign key / cascade	A column pointing to a row in another table; cascade deletes children with the parent.	Deleting a user deletes their transactions.
Validation	Checking input before using it.	A negative goal amount is rejected with a message.
Authentication	Proving who you are.	Logging in with email and password.
Password hash (scrypt)	A one-way scrambling of a password; the original cannot be recovered.	Only the hash is stored in users.password_hash.
JWT	JSON Web Token: a signed text that proves you logged in; the server checks the signature.	Sent as `Authorization: Bearer eyJ…`
CORS	A browser rule: a site may call an API on another domain only if that API allows it.	The Render API allows the Vercel site.
Environment variable	A setting given to a program from outside its code.	DATABASE_URL, JWT_SECRET_KEY.
Docker / container	Packages an app with everything it needs so it runs the same everywhere.	The API image contains Python, packages, code and trained models.
docker compose	Starts several containers together.	Database + API + web server with one command.
nginx / reverse proxy	A web server that serves files and forwards some URLs to another server.	nginx forwards /api/… to the Flask container.
gunicorn	A production server that runs Python web apps with several workers.	Runs Flask on Render.

Term	Meaning	Example in FinTwin
CDN	Content Delivery Network: copies of static files on servers worldwide.	Vercel serves the React build.
Digital Twin	A virtual model of a real system kept in sync with it, used to analyse and test changes.	FinTwin's TwinState is rebuilt from the user's data.
Synthetic data	Realistic but computer-generated data.	The demo account's 24 months of transactions.
Seed (data / random)	Data seed: initial data loaded into a database. Random seed: a number that makes random generation repeatable.	`flask seed`; seed=2026.
Idempotent	Running something twice has the same effect as once.	A second `flask seed` changes nothing.
pandas DataFrame	A table in Python memory with rows and columns.	All of a user's transactions for calculations.
Machine learning model	A program that learned patterns from examples and makes predictions.	The XGBoost forecaster.
Training / inference	Training = learning from data (offline). Inference = using the trained model.	`flask ml train` vs. a forecast request.
Baseline	A simple method that a smarter model must beat to be worth using.	"Next month = this month".
MAE / R^2	MAE: average size of the error. R^2: how much of the variation the model explains (1 = perfect).	XGBoost MAE ₹5,312, R^2 0.87.
XGBoost	A gradient-boosted decision-tree algorithm, strong on tabular data.	Spending forecaster.
Isolation Forest	An unsupervised anomaly detector: unusual points are separated by random splits quickly.	Flags an unusually large purchase.
Robust z-score (MAD)	How far a value is from the median, measured in a way outliers cannot distort.	$z \geq 3.5$ marks an unusual amount.
TF-IDF	Turns text into numbers by word importance (frequent here, rare elsewhere).	Used by the categorizer and retrieval.
Embedding / LSA	A vector of numbers representing meaning; LSA builds them by compressing TF-IDF.	"rainy day savings" lands near "emergency fund".
RAG	Retrieval-Augmented Generation: fetch trusted passages first, then answer using them.	The coach cites "Emergency funds".
LLM	Large Language Model, e.g. GPT or Gemini, which generates text.	Optional coach provider.
Hallucination	When an AI states something false as if true.	Prevented by the grounding check.
Grounding check	Verifying that every number in an AI answer exists in the provided data.	An answer with an unknown ₹92,640 is rejected.
Deterministic	Same input always gives the same output.	The What-If engine.
EMI / SIP	EMI: fixed monthly loan repayment. SIP: fixed monthly investment into a fund.	₹10,000 SIP in the demo account.
Debt-to-income (DTI)	EMIs divided by income.	₹30,000 EMIs on ₹1,00,000 income = 30%.
Emergency fund	Savings kept for emergencies, measured in months of essential costs.	3–6 months is recommended.
Unit test / coverage	Small automatic checks of one piece of code; coverage = share of code the tests run.	503 backend tests; about 98% coverage at last measurement.

12. Project Viva Questions and Answers

Short, accurate answers you can say aloud. Numbers come from the project's measured results (docs/ml.md and the metrics files) and its tests.

Q1. What is the purpose of this project?
FinTwin helps people understand and plan their finances. It builds a digital twin of the user's finances from their transactions and profile, then scores financial health, predicts spending, detects risks, simulates decisions before they are made, plans goals, and explains everything through an AI coach that never invents numbers.

Q2. What is a Digital Twin, and why is FinTwin one?
A digital twin is a virtual model of a real system that stays in sync with it and can be used to predict and test changes safely. FinTwin's TwinState mirrors the user's real data, is rebuilt after every change (including the monthly history), and the simulator copies it to test "what if" changes.

Q3. Why not just build an expense tracker?
A tracker only records the past. FinTwin answers decision questions: how healthy am I and why, where is spending heading, can I afford this, is my goal realistic. The tracker part is only the input layer.

Q4. Why did we choose this technology stack?
React is the most common UI library and suits a component-based dashboard; Vite makes development fast. Flask is small and easy to explain, and Python is required anyway for pandas, scikit-learn and XGBoost. PostgreSQL gives reliable transactions and constraints. Docker makes the whole stack start with one command. Each choice keeps the project understandable for a college team.

Q5. How was the website developed?
In 14 phases, from architecture to deployment, each ending with tests, fixes, documentation and a git commit. Calculations were built and unit-tested before the frontend, then the whole app was tested end to end against 18 acceptance criteria, then hardened with edge-case tests.

Q6. How does the frontend communicate with the backend?
Over HTTPS with JSON. One module, api.js, sends every request with the login token in the Authorization header and unwraps the {data, meta} or {error} response. In development Vite forwards /api to Flask; in Docker nginx does; when hosted separately the frontend calls the Render URL and CORS allows it.

Q7. Where does the demo account data come from?
From the seed command, flask seed, which Docker runs at start-up. It creates a fictional user and generates 24 months of transactions with a synthetic generator (fixed random seed 2026), plus a fixed profile, an education loan, three investments and three goals written as constants in seed/demo.py. Everything is stored in PostgreSQL and read back through the normal API.

Q8. Is the data real, synthetic, hard-coded or API-based?
Synthetic and stored in the database. The transactions are generated; some starting values are constants in the backend seed code; nothing is hard-coded in the frontend and nothing comes from an external API. Real users' data comes from their own manual entries and CSV uploads, and goes through the same code.

Q9. How is data stored and retrieved?
Stored in 15 PostgreSQL tables through SQLAlchemy models, created by Alembic migrations. Repositories read a user's rows (always filtered by their user id) into pandas DataFrames; services calculate; routes return JSON. Derived history (snapshots, scores, alerts) is recomputed and stored after every write.

Q10. How does authentication work?

Passwords are stored as scrypt hashes. Login returns a JWT valid for 12 hours, signed with a secret from an environment variable. The browser sends it with every request; the API verifies the signature and takes the user id from it. A 401 sends the user back to the login page.

Q11. What happens when the backend or an API fails?

The app degrades instead of crashing: an unreachable server shows a clear message with Try again and keeps the session; a database outage returns 503 and /api/health reports it; missing ML models fall back to baselines and rules; any LLM failure, or an answer with an unverifiable number, falls back to the deterministic local answer; a crashing page is contained by an error boundary.

Q12. How is the Financial Health Score calculated?

Seven components (savings 20, emergency fund 20, debt 20, cash-flow stability 15, investments 10, goals 10, spending behaviour 5) each turn one ratio into 0–100 with a curve anchored on guidelines such as the 50/30/20 rule and a 3–6 month emergency fund. The total is the weighted mean; components that can't be measured are excluded and the weights renormalised.

Q13. How does forecasting work, and why XGBoost?

One global XGBoost model is trained offline on 300 synthetic users, because one user has too few months to train on. Each series is scaled by the user's own level. XGBoost is strong on tabular lag features, fast and explainable. It is used unless a simple baseline was 20% better on that user's last 6 months. On 60 unseen users it reached MAE ₹5,312 and R^2 0.87, against ₹6,975 and 0.78 for the best baseline.

Q14. How does anomaly detection work?

Unsupervised: a robust z-score per category (median and MAD) plus a per-user Isolation Forest. Both agreeing gives HIGH, the rule alone MEDIUM. Also duplicates, abnormal months with a same-month-last-year check (so festive months aren't flagged every year) and category drift. Measured on planted anomalies: F1 0.79.

Q15. How does the What-If simulation work, and why is it deterministic rather than LLM-generated?

It projects cash month by month twice, without and with the change, and compares the results at 12 months. It is deterministic so the numbers are correct, reproducible, testable against hand calculations and explainable. LLMs can produce plausible but wrong arithmetic; in FinTwin the LLM may only explain the engine's numbers.

Q16. What is the difference between prediction and simulation?

Prediction (forecasting) estimates what will probably happen if behaviour continues, learned from history, with an uncertainty range. Simulation computes what would happen under stated assumptions, exactly and reproducibly. The simulator deliberately uses averages, not ML forecasts, as its baseline.

Q17. What is RAG and why use it?

Retrieval-Augmented Generation: before answering, retrieve relevant passages from trusted documents and answer from them. FinTwin retrieves from 13 guides written for the project using TF-IDF plus LSA embeddings (hit@3 100% on 20 test questions), so general advice is sourced and cited, not taken from the model's memory.

Q18. How does the AI coach access user data, and how is hallucination prevented?

The coach calls the same services the app uses and passes only the needed, verified sections as context; the LLM never queries the database. Numbers are computed before any LLM is involved, and a grounding check rejects any answer containing a ₹, % or decimal not found in the context (1% tolerance), replacing it with the deterministic local answer.

Q19. What makes this an AI/ML project?

Supervised ML (XGBoost forecaster, TF-IDF + Logistic Regression categorizer), unsupervised ML (Isolation Forest), NLP and information retrieval (entity parsing, RAG), optional generative AI with verification, and an ML pipeline with evaluation against baselines and documented results.

Q20. How was the ML evaluated without real data?

On a synthetic population with known ground truth: users split 80/20 so models are scored on people they never saw, forecasts scored on held-out final months, the categorizer scored on merchants it never saw (grouped cross-validation), and anomaly detection scored against planted anomalies. The limitation, that synthetic data is partly circular, is stated openly.

Q21. How does the CSV import work?

Two steps. Preview parses and validates every row (dates, ₹ amounts, required fields), auto-categorizes it and flags duplicates without saving. The user reviews and confirms; the server re-validates and saves all rows in one transaction. It tolerates real bank formats such as account lines above the header, ; delimiters and 0.00 in the unused debit/credit column.

Q22. What security measures are implemented?

Hashed passwords, expiring signed tokens, a refusal to start in production with a default secret, per-user data scoping with 404 for others' records, strict input validation, ORM queries, upload size and row limits, restricted CORS, secrets only in environment variables, and a non-root Docker container.

Q23. How is the project tested?

503 backend tests (about 98% coverage at last measurement): formulas against hand calculations, the twin builder, score, forecasting, anomalies, simulations, goals, coach NLU and grounding, CSV edge cases, every API route with junk input and without a token, and the whole pipeline on leap days and month-ends. Plus 5 frontend tests, linters and a full end-to-end walkthrough in the browser.

Q24. How is the project deployed?

Locally with one command, docker compose up --build, which was verified on a clean copy. For hosting, the repository contains a Render blueprint for the API (Docker) and a Vercel config for the frontend, with Neon as the database; GitHub pushes trigger deployments.

Q25. How could the project be scaled for more users?

The API is stateless (JWT), so more gunicorn workers and more containers behind a load balancer work directly. Next steps: move recalculation and anomaly-model refits to a background job queue, cache computed twins, add database indexes and connection pooling, serve the frontend from a CDN (already the case on Vercel), and use a managed, scalable PostgreSQL.

Q26. What are the project's limitations?

Synthetic training data; about 62% categorization accuracy on unseen merchants; forecasts ignore raises and bonuses; no taxes or market volatility in simulation; rule-based intents; untested live LLM calls; no bank integration, password reset or rate limiting; a shared demo account; free-hosting cold starts.

Q27. What would we change or improve in the next version?

Real consented data and live accuracy monitoring; Account Aggregator import; background workers; cookie-based sessions with rate limiting and password reset; per-visitor demo accounts; Monte-Carlo simulation; better embeddings; tax and retirement planning; a mobile app.

Q28. Why PostgreSQL rather than a NoSQL database?

Financial data is relational (users, transactions, categories, loans) and needs exact decimals, constraints and transactions (an import is all-or-nothing). JSON columns cover the few truly variable parts.

Q29. Why is income stored as a transaction rather than a separate table?
One ledger means cash flow is always income minus expenses over one table, so nothing can be double-counted between two sources.

Q30. What happens if a user has very little data?
The twin uses the declared profile values until a full month of transactions exists, and says so; forecasts report insufficient data or low confidence; health-score components that cannot be measured are excluded instead of scored as zero.

Q31. Is FinTwin giving financial advice?
No. It is an educational decision-support tool. It uses words like "estimated" and "projected", shows a disclaimer, never recommends specific securities, and redirects stock-picking questions to a SEBI-registered adviser.

12.1 A five-minute demonstration script

1. **Dashboard** (demo account): the score, KPIs and alerts. Say: every number comes from the API.
2. **Digital Twin**: observed vs declared sources, "how it's calculated", predicted state.
3. **Transactions → Import CSV**: download the sample statement and upload it; show the per-row status and auto-categories; correct one category.
4. **Forecasts**: the chosen model, its backtest and the likely range.
5. **Simulator**: a ₹4,00,000 purchase → CRITICAL, with the month cash runs out and the assumptions (₹1,30,000 is LOW for the demo account when checked).
6. **Coach**: "Can I afford a laptop for 1.3 lakh?", then "Which stock should I buy?" (educational redirect).
7. **Risks**: a planted anomaly with its explanation and recommended action.

> **TRY IT** Before a demonstration on the hosted site, open it a few minutes early to wake the free API instance, and consider resetting the shared demo account so earlier visitors' edits are gone.